



Departamento de Computación

Asignatura: Ingeniería de Software I (3385)

Año 2025

PRÁCTICO 2B: EJERCICIO TRANSVERSAL INTEGRADOR: SISTEMA DE GESTIÓN ESTUDIANTIL

En el Práctico 1B, cada equipo realizó un análisis del sistema de gestión estudiantil, generando los requerimientos iniciales, un diagrama de clases de contexto y un backlog. Dando continuidad a eso, el Práctico 2B se enfoca en aplicar conceptos de diseño de software y comenzar la implementación, utilizando un código base que les proporcionaremos.

Objetivos del práctico:

Que las/os estudiantes puedan:

- Comprender y analizar una arquitectura de software basada en un código base existente.
- Identificar y explicar patrones de diseño implementados en un sistema real (Ingeniería Inversa).
- Diseñar y modelar flujos de interacción a nivel de componentes tecnológicos.
- Implementar funcionalidades específicas del backlog priorizado e integrarlas al código existente.
- Consolidar la colaboración en equipo y el uso de herramientas de control de versiones.

Diseño, patrones y primeros pasos en la Implementación

En esta etapa es fundamental que comprendan la diferencia entre la metodología Scrum, que nos indica *cómo* organizar y gestionar el trabajo, y el *set de herramientas* o tecnologías, que se usan para llevarlo a cabo.

El código base que proporcionaremos desde la asignatura incluye la mayoría de las *herramientas* ya configuradas y funcionando para las funcionalidades básicas de login (inicio de sesión) y gestión de usuarios.

Actividades a realizar

- **Preparación del Código Base:** Desde el repositorio de la asignatura https://github.com/marcelouva/is1_2025_eti.git, cada equipo deberá realizar un **fork** del proyecto base del sistema de gestión estudiantil, para crear una copia independiente del código en su propio repositorio.

Este repositorio incluye la estructura inicial del proyecto en Java, las configuraciones de Maven y una versión funcional del sistema de login y carga básica de usuarios.

Luego, cada integrante del equipo deberá clonar ese repositorio en su entorno local y verificar que el proyecto compile y se ejecute correctamente.

- **Análisis del código fuente y patrones de diseño:** Dentro del código base provisto, deberán identificar y analizar el uso de un patrón de diseño que ya se encuentra implementado.

Expliquen:

¿Qué patrón identificaron?

Identificamos el uso del patrón Singleton.

¿Dónde y cómo se aplica en el código?

El patrón se aplica en la clase DBConfigSingleton.java. Se aplica al definir el atributo instance del tipo DBConfigSingleton como private static, y al definir un constructor como private, lo cual evita la posibilidad de instanciación directa. La instanciación sólo es posible a través del método getInstance, el cual tiene visibilidad public static, y permite la creación de una, y sólo una, instancia de la clase DBConfigSingleton.

¿Qué problema resuelve este patrón en ese contexto?

El patrón resuelve el problema de asegurar que sólo haya una única conexión a la vez a la base de datos desde un mismo sistema.

- **Implementación de Historias de Usuario (HU):** Cada equipo deberá implementar la siguiente Historia de Usuario en el proyecto.

ID de HU	001
Título	Alta de profesor al sistema
Declaración	Como administrador del sistema, quiero registrar un nuevo profesor ingresando su información personal, para poder asignarlo a las asignaturas correspondientes dentro de una carrera.
Descripción detallada	Cuando se quiere registrar un nuevo profesor, se debe mostrar un botón que permita acceder al registro de un nuevo profesor. Tras hacer click en él se debe mostrar un formulario, que presente cuadros de diálogo para ingresar la información del profesor, indicando cuáles campos son obligatorios y cuáles son opcionales. Si se completaron, al menos, todos los campos obligatorios, y de forma correcta, debe registrar el nuevo profesor y mostrar un mensaje indicando que el registro del profesor fue exitoso. Si falta de completar alguno de los campos obligatorios, debe impedir el registro del nuevo profesor indicando que debe completarse el campo faltante. Si el campo DNI y/o el campo Correo Electrónico no son completados correctamente, debe impedir el registro del profesor y mostrar un mensaje indicando como debe completarse el campo respectivo. Si el DNI y/o el Correo Electrónico ya se encuentran registrados en el sistema, debe impedirse el registro del nuevo profesor, y mostrar un mensaje indicando que el DNI (o el Correo Electrónico) ya se encuentra registrado en el sistema. Debe mostrar un botón Cancelar que permita salir del formulario de registro de profesor y volver al panel de control del usuario.
Criterios de Validación	• Flujo exitoso: Al completar todos los campos obligatorios (nombre, apellido, correo, DNI) con datos válidos y guardar, el sistema muestra

(Criterios de Aceptación)	un mensaje de éxito. - Validaciones de Datos: El sistema debe impedir el registro si: - Faltan campos obligatorios. - Si el formato del correo electrónico no es válido. - El correo electrónico o el DNI ya existen en la base de datos. - Manejo de Errores: Si alguna validación falla, el sistema debe mostrar un mensaje de error claro, sin permitir que se guarde el formulario. - Acción de Cancelar: El formulario debe incluir un botón "Cancelar" que elimine todos los datos ingresados y devuelva al usuario a la pantalla anterior.
Tareas asociadas a la implementación	- Desarrollar la interfaz de usuario (templates de dashboard, login, formularios de registro de usuarios y de profesores, mensaje de error). - Crear las tablas <i>users</i>, <i>persons</i> y <i>professors</i> en la base de datos. - Implementación de la conexión de la aplicación con la base de datos. - Implementación de las clases del modelo (<i>user</i>, <i>person</i> y <i>professor</i>). - Implementar los formularios de registro de usuario y registro de profesor.